

Test-rig Documentation

1. General Instructions

- **Individual build.** Every member builds their own project end to end. Discussion and debugging help between members is encouraged, but the hardware, firmware and documentation must be your own work.
- **Allocation.** Fill in your preference in the Google Form. Slots will be allotted based on your preference order, subject to the vacancies available per topic.
- **Final review.** A review will be conducted at the end of the timeline. You will demonstrate a working rig and present what you built, what you learned, what failed, and how you fixed it. A partially working rig with a clear explanation of the failure will be scored above a working rig you cannot explain.
- **Documentation is mandatory.** Maintain a GitHub repository under the RMI organisation containing your firmware, CAD files, wiring diagram, a README explaining how to run the project, and a short build log with the problems you hit and how you solved them. Commit regularly through the seven weeks — a single commit at the end will not be accepted as documentation.
- **Work within existing inventory.** Use the components and 3D printing facilities already available in the lab. Any request for a new purchase must be raised early, justified, and approved before ordering.
- **Core vs. Stretch.** Each topic lists **Core Objectives** (required for the review) and **Stretch Objectives** (bonus). Get the core working first; stretch goals only count once the core is demonstrably complete.

2. Topics

2.1 Turret Mechanism

Problem Statement

Design and build a 2-DOF turret (yaw and pitch) capable of launching a projectile. The turret is commanded not by buttons or a thumbstick, but by an **orientation-based joystick** that you design yourself — a handheld unit built around an ESP32 and an IMU, where the physical tilt and rotation of your hand maps directly to the pose of the turret. Point the controller, and the turret points with it.

Core Objectives

- Build the handheld controller yourself: ESP32 + IMU + whatever supporting electronics and enclosure you need. It must be self-contained enough to be held and moved freely in one hand.
- Estimate the controller's orientation from raw IMU data. **Write the fusion filter yourself** — a complementary filter or Madgwick/Mahony implementation. Dropping in a pre-built library that outputs Euler angles for you does not count.
- Implement a startup calibration routine that measures and removes gyro bias, so the turret does not drift when the controller is held still. Demonstrate this: hold the controller stationary for 60 seconds and show the turret does not wander.
- Map controller orientation to turret yaw and pitch with a defined workspace (e.g. $\pm 90^\circ$ yaw, $0\text{--}60^\circ$ pitch). Enforce these limits **in firmware**, and add a small deadband and slew-rate limit so the turret moves smoothly instead of jerking on sensor noise.
- Send commands wirelessly from the controller to the turret over ESP-NOW or Wi-Fi. Both ends run ESP-IDF.
- Implement a **failsafe**: if the turret receives no packet for more than ~ 500 ms, it must hold its last commanded position and refuse to launch until the link is restored.
- The launch trigger must be an explicit, deliberate input on the controller — not something that can fire from an accidental gesture.
- CAD the full turret assembly and the controller enclosure. Both must be 3D-printable and must actually be printed and assembled. Justify your choice of pivot arrangement, motor/servo mounting, and how you constrained backlash.

Performance Targets

- **Static accuracy**: commanded vs. achieved angle within $\pm 3^\circ$ across the workspace.
- **Latency**: visible response to controller movement in under ~ 100 ms end to end.
- **Repeatability**: return to the same commanded pose five times and show the spread.

- **Hit test:** engage targets at three marked distances. Report hits out of attempts.

Stretch Objectives

- **Ballistic aim assist:** given a target distance entered by the operator, have the firmware compute and set the required pitch angle from a projectile-motion model, then compare predicted vs. actual landing point and report the error.
- **Telemetry:** stream commanded angle, measured angle and error over serial or Wi-Fi, and include step-response plots in your documentation. Use the plots to justify your gains rather than tuning by feel.
- **Homing on power-up:** use limit switches or a hard stop to establish a known zero at boot instead of assuming the turret starts centred.

Learning Outcomes

IMU sensor fusion and filtering, sensor calibration and bias estimation, wireless embedded communication and failsafe design, ESP-IDF development, CAD modelling and mechanism design.

2.2 Line Follower

Problem Statement

Build a differential-drive robot that follows a black line to the finish using an IR sensor array. The track is deliberately hostile: sharp corners, tight bends, and junctions that will throw off a naive controller. The run has **two phases** — an exploration round in which the robot discovers the track, and a second round in which it must take the **optimal path** it worked out. You are free to add any other sensors from the existing inventory that help your robot.

Hard-coded solutions will NOT be accepted. Any robot whose path is a pre-written sequence of turns will be disqualified. Your robot will be tested on a track layout you have not seen, and may be tested on a mirrored version of the practice track.

Core Objectives

- Implement a **sensor calibration routine** that sweeps the array across line and background at the start of every run, records per-sensor min/max, and normalises readings. Do not hard-code thresholds — they will not survive a change in lighting or

surface.

- Compute line position as a **continuous weighted estimate** from the sensor array, not a discrete left/centre/right decision.
- Implement PID control on line position, including **integral anti-windup** and a sensible derivative filter. Document your tuning process with data, not adjectives.
- Detect and correctly classify junctions — left, right, T, cross, dead end — and handle each. Junction detection must be robust at speed, not only when crawling.
- **Line-loss recovery**: if the line is lost, execute a defined search behaviour and reacquire it. The robot must not simply drive off the table.
- During the exploration round, build an internal representation of the track (path string or graph). After exploration, run a **path optimisation algorithm** — LSRB reduction, BFS or Dijkstra — and demonstrate that the second run is measurably shorter than the first. Print or log the discovered path and the optimised path as evidence.
- Both **speed and accuracy** are evaluated. A fast robot that leaves the line scores below a slower one that completes cleanly.
- CAD and 3D-print the chassis. Sensor mounting height and spacing must be a deliberate, justified choice — explain it in your documentation.
- Firmware in ESP-IDF.

Performance Targets

- Complete the exploration round with no manual intervention.
- Optimised run must be at least 20% faster than the exploration run on the same track.
- Three consecutive optimised runs without leaving the line.

Stretch Objectives

- **Speed-adaptive control**: reduce speed automatically on high-curvature sections and accelerate on straights, using the error signal itself to infer curvature.
- **Battery compensation**: read battery voltage and scale motor commands so behaviour stays consistent as the pack drains — a common and rarely-diagnosed source of "it worked yesterday".
- **Odometry**: if encoders are available in inventory, estimate distance travelled per segment and use it to weight the path optimisation by actual length rather than segment count.
- **Live telemetry** of error, PID terms and motor output over Wi-Fi, for offline tuning analysis.

Learning Outcomes

PID control and controller tuning from data, sensor calibration and normalisation, maze-solving and shortest-path algorithms, state machines for robot behaviour, ESP-IDF development, CAD modelling.

2.3 Intelligent Sorting System

Problem Statement

Build a system that sorts nuts and bolts by size. Design a feeding mechanism that presents items from a mixed bag onto an inspection platform one at a time. A **phone camera** images each item, the frame is streamed to a laptop for processing, and the classified item is directed into the correct bin: **Nut, Bolt, or Misc**. Running count per category is displayed live on a dashboard.

No embedded camera. The vision input must come from your phone camera, streamed to your laptop for processing. *Hint: look into methods for sending your phone camera feed to a laptop over the network.* An ESP32 cannot run image processing of this weight — the ESP32's job is mechanism control and talking to the laptop.

Core Objectives

- **Autonomy:** after the mixed bag is loaded, human intervention must be minimal to none. Singling out items, imaging, deciding and diverting all happen without a hand in the loop.
- **Feeding mechanism:** design and 3D-print a mechanism that reliably separates one item at a time from a mixed pile. This is the hardest mechanical part of the project — vibratory, rotating-disc, escapement, and inclined-belt approaches are all valid. Justify your choice and document what you tried first and why it failed.
- **Jam detection and recovery:** the feeder *will* jam. Detect a stall or a timeout with no new item, and execute an automatic recovery (reverse, agitate, retry) before giving up and flagging the operator.
- **Classification into three classes** — Nut, Bolt, Misc. Choose either a classical CV pipeline (threshold, contours, shape descriptors, aspect ratio) or a trained model, and **justify the choice** against your accuracy and latency requirements.
- **Confidence handling:** when the classifier is not confident, the item goes to Misc rather

than being guessed into Nut or Bolt. Define and state your confidence threshold.

- **Dashboard:** live running count per category, current system state (idle / feeding / imaging / sorting / jammed), and a session log exportable as CSV.
- **Accuracy over speed.** A slow, correct sorter beats a fast, confused one.
- ESP32 handles the mechanism and communicates with the laptop. Firmware in ESP-IDF.

Performance Targets

- **Test set:** run a mixed bag of at least 50 items, including at least 5 deliberate "Misc" objects (washers, odd screws, debris).
- **Accuracy:** report a full **3×3 confusion matrix** and per-class precision and recall. A single "accuracy percentage" is not an acceptable result.
- **Robustness:** repeat the run under a noticeably different lighting condition and report how the numbers change.
- **Throughput:** report items sorted per minute, and the number of jams and successful auto-recoveries over the run.

Stretch Objectives

- **Touching objects:** handle two items arriving overlapping or in contact, using contour separation or watershed, instead of failing on them.
- **Size discrimination via calibration:** place a reference object of known dimension in the frame to establish a pixels-per-millimetre scale, then classify by real physical size (e.g. M4 vs M6) rather than pixel size. The system must not break when the phone is remounted at a slightly different height.
- **Finer classes:** extend beyond three bins to sort by thread size within the Nut and Bolt categories, using your millimetre calibration.
- **Failure gallery:** log every misclassified frame to disk with its predicted and true label, and include an analysis of the failure modes in your documentation. This is worth more than a marginally higher accuracy number.
- **Remote start/stop** from the dashboard, with a software emergency stop that halts the mechanism immediately.

Learning Outcomes

Computer vision and classification pipeline design, camera calibration and metric measurement from images, quantitative evaluation of a classifier, IoT and networked systems, mechanism

design for singulation and feeding, fault detection and recovery, ESP-IDF development, CAD modelling.

3. What Every Submission Must Include

Deliverable	Detail
Working rig	Demonstrated live at the final review, meeting the Core Objectives of your topic.
GitHub repository	Under the RMI organisation. Firmware, CAD files, wiring diagram, README with setup and run instructions.
Build log	Committed progressively across the seven weeks. What you tried, what broke, what you changed.
Measured results	The Performance Targets for your topic, reported as numbers and plots — not claims.
Presentation	Short walkthrough of the design decisions, the failures, and the learning.